

# Code Localization as Sequential Set-Coverage: A Trainable Multi-Round Scaffolding for Small LLMs

Anonymous ACL submission

## Abstract

Before a coding agent can fix a GitHub issue, it must localize the files and functions to change; a mistake here cannot be undone by any later reasoning. Strong localization scaffoldings such as OpenHands and SWE-Agent are built for long contexts and large models, so the open 7–14B code models most teams can run overflow a 32K-token budget before the search ends. Training small models is no easier: a multi-step agent yields only a sparse terminal reward, forcing expensive reinforcement learning. We present a multi-round localization scaffolding that removes both barriers. The agent refines a fixed-length candidate list over rounds, each round inspecting a bounded window of ranked nodes and their neighborhood on a code graph, so per-call token cost stays roughly constant. The round structure also exposes a dense per-round target: under potential-based shaping, the per-round signal telescopes to end-to-end recall@ $L$ . The agent therefore trains by ordinary supervised fine-tuning—a weighted cross-entropy with a differentiable coverage term—rather than sparse-reward RL. This recasts localization from ranking to sequential set-coverage on a code graph. Trained this way, small open models become competitive with GPT Codex 5.3 under the same scaffolding, and a 30B model matches open-ended OpenHands agents at a fraction of the token cost, on SWE-Bench Lite, SWE-Bench Verified, and the multilingual SWE-PolyBench.

## 1 Introduction

LLM coding agents increasingly resolve real GitHub issues by editing a repository directly, and benchmarks such as SWE-Bench now track how often they succeed (Jimenez et al., 2024b). A patch can only be correct, however, if it changes the right code. Before any edit, the agent must identify the files and functions an issue actually touches, the step known as *code localization*. This step is decisive and largely unrecoverable: once a wrong

location enters the agent’s editable context, the repair stage reasons over the wrong code, and no later reasoning recovers the correct patch (Jimenez et al., 2024b; Yang et al., 2024a). The strongest systems are built around this fact. Xia et al. (2024a) drop agentic interaction and resolve issues with a fixed localize-then-repair pipeline, crediting much of their accuracy to a hierarchical localization phase, and methods that improve the localization stage report corresponding gains in end-to-end resolution (Sohrabizadeh et al., 2025). Localization is therefore not a preprocessing convenience but the step that determines how well downstream repair can perform.

Yet localizing well is hard: a real repository holds thousands of files joined by tangled import, call, and inheritance edges, while an issue report describes symptoms in natural language that can sit far from the code that must change. Current systems close this gap in four ways. Reinforcement learning methods train the search policy itself: RepoSearcher pairs tool-integrated RL with rejection-sampled supervised fine-tuning (Ma et al., 2025), and SWE-RL scales RL over open software-evolution data (Wei et al., 2025). General-purpose agent scaffoldings give a capable LLM an open-ended action space with shell access, as in SWE-Agent and OpenHands (Yang et al., 2024a; Wang et al., 2025). Graph-exploration agents such as LocAgent let an LLM traverse a repository code graph, hopping freely between related code units (Chen et al., 2025b). Dense retrieval methods skip the agent altogether: SWERank scores code units with a bi-encoder retriever and a listwise LLM reranker (Reddy et al., 2025b).

The first three families pay for accuracy in tokens, and none of them bounds the cost. An RL agent, an open-ended scaffold, and a graph explorer all keep issuing tool calls and appending observations until they choose to stop, so a localization trajectory grows without a predictable ceiling. Under

an OpenHands-style scaffolding, a single localization run can exceed 131K tokens (Section 4.4). A model with a long context window and ample compute absorbs that cost; the open 7–14B code models most teams can realistically run do not. A Qwen-class coder with a 32K-token window overflows before the search terminates. Reinforcement learning compounds the problem: a multi-step episode returns only a sparse terminal reward, so adapting a small model to it takes many long and expensive rollouts. Dense retrieval is the one cheap option, but a single-step, order-centric ranker cannot revisit a discarded candidate or follow a structural edge, so it leaves the sequential, graph-structured side of the problem untouched. No existing method is at once bounded in token cost at inference and cheap enough to train on a small open model.

## 2 Related Work

**Reinforcement learning for localization.** RepoSearcher equips a repository agent with a curated tool set and trains it through rejection-sampled supervised fine-tuning followed by tool-integrated RL, reaching strong localization with a 32B model (Ma et al., 2025). SWE-RL scales RL over open software-evolution data and shows the learned reasoning transfers across software tasks (Wei et al., 2025). RepoNavigator narrows the agent to a single jump-to-definition tool and trains end-to-end with RL from a base model, so that a 7B agent can outperform substantially larger baselines (Zhang et al., 2025). In every case training optimizes multi-step episodes that return only a sparse terminal reward, so adaptation is costly in rollouts and compute. Our round decomposition exposes a dense per-round target that telescopes to recall@L (Section 3.3), turning the same problem into ordinary supervised fine-tuning.

**Agentic scaffolding.** SWE-Agent and OpenHands wrap a strong LLM in a general-purpose environment with shell access and an open-ended action space, and they set much of the standard for end-to-end issue resolution (Yang et al., 2024a; Wang et al., 2025); Moatless provides a comparable tool-driven loop (Örwall, 2024). Agentless instead drops interaction and runs a fixed localize-then-repair pipeline, showing that a simple prompted procedure can rival full agents (Xia et al., 2024a). The interactive scaffoldings place no ceiling on trajectory length, so token cost is unbounded and a localization run easily overflows the context window

of the open 7–14B models we target. Agentless is bounded, but its localization is a fixed prompted pipeline, not a component a small model can learn to improve. Our scaffolding bounds the token cost of every round and stays trainable, so a small model can run the search within budget and be trained to localize better.

**Graph-based exploration.** A related line hands the LLM the repository’s structure. LocAgent builds a heterogeneous code graph and lets the model explore it through graph-traversal actions (Chen et al., 2025b), CoSIL has the model search a call graph it expands on the fly without a pre-built index (Jiang et al., 2025), and RepoGraph supplies a repository-level code graph for software-engineering agents (Ouyang et al., 2025). This work establishes that structural edges carry localization signal. The traversal, however, is unconstrained, so trajectory length and token cost again grow without a bound. We use the same structure through bounded neighborhood expansion around ranked seeds, so graph signal raises recall without inflating the budget.

**Retrieval and reranking.** Localization is also posed as retrieval. Lexical matching (Robertson et al., 1994) and learned code embeddings (Feng et al., 2020; Guo et al., 2021; Zhang et al., 2024a) score code units against the issue text, and SWERank sharpens this for issue localization with a bi-encoder retriever and a listwise LLM reranker (Reddy et al., 2025b); CoRet improves the retriever for code-editing queries (Fehr et al., 2025) and SpIDER incorporates repository structure into the dense representation (Chaudhari et al., 2025). These rankers are cheap and bounded, but the objective is per-query and order-centric, optimizing the order of one list rather than coverage of the gold set, and a single pass cannot revisit a discarded unit or follow a structural edge. SWERank+ adds a multi-turn agentic loop with a memory buffer (Reddy et al., 2025a), yet the loop is open-ended and the training signal stays a ranking objective. We recast localization as sequential set-coverage on a code graph and train an objective provably co-optimal with recall@L (Section 3.4), keeping the bounded cost of retrieval while adding the set recovery and multi-round refinement that ranking omits.



Figure 1: The multi-round localization scaffolding. Each round consumes a bounded window of  $K$  ranked candidates, expands neighborhoods around  $C$  seed centers via bounded BFS on the code graph, and refines a fixed-length returned list  $L_r$ . Per-call token cost stays roughly constant across rounds.

### 3 Method

#### 3.1 Problem Setup

#### 3.2 Multi-Round Localization Scaffolding

The scaffolding processes a single issue  $q$  over  $R$  rounds. A retriever (dense, lexical, or their union) returns a ranked list of code units, and each round consumes the next  $K$  unseen entries of that list. At the start of round  $r$ , the scaffolding holds three pieces of state: the current returned list  $L_{r-1}$  (initialised to  $\emptyset$  at  $r=0$ ), the set of node identifiers  $\mathcal{S}_{r-1}$  already shown to the model, and an offset  $rK$  into the ranked list. The loop terminates when the ranked list is exhausted or a maximum number of rounds  $R_{\max}$  is reached. An optional confident-stop hook is left for the RL stage (Section 5); the supervised training of Section 3.4 does not predict it, so a trajectory always runs to  $R_{\max}$  or exhaustion.

Each round combines a bounded window of newly-ranked candidates with a bounded graph-neighborhood expansion, then refines  $L$ ; Figure 1 sketches the loop.

**Window.** The scaffolding pulls the next  $K$  unseen entries of the ranked list starting at offset  $rK$ , skipping any node already in  $\mathcal{S}_{r-1}$ . We denote this window  $W_r$ . If  $W_r$  is empty the round terminates the loop.

**Step A: seed-center selection.** A single model call selects  $C$  seed centers from  $W_r$ , i.e. nodes whose graph neighborhoods are most likely to contain unseen gold units. If the model returns fewer than  $C$  valid identifiers we pad with the top-ranked nodes of  $W_r$  not yet returned, so Step A always emits exactly  $C$  centers.

**Step B: neighborhood expansion.** For each of the  $C$  centers, the scaffolding runs a depth- $d$  BFS

along structural edges of the code graph (*contains* by default; ablations in Section 4.3 cover the four edge types). The neighbors are filtered to the next  $N$  unseen entries of the ranked list, so the candidate set the model sees stays bounded. A model call per center then selects the subset of these neighbors that must be modified, and the union across centers,  $\mathcal{N}_r$ , is deduplicated.

**Step C: list refinement.** A final model call sees the issue  $q$ , the current list  $L_{r-1}$ , the window  $W_r$ , and the deduplicated set  $\mathcal{N}_r$ , and selects  $L$  identifiers from  $L_{r-1} \cup W_r \cup \mathcal{N}_r$  to form  $L_r$ . The scaffolding then records the newly-shown identifiers, advances the offset, and continues.

Three properties of this loop matter for what follows. *First*, every model call sees only  $W_r$ , a single neighborhood, or the deduplicated round pool, so the per-call token cost is roughly constant across rounds; a 32K context comfortably absorbs each call regardless of how many rounds the trajectory uses. *Second*, the loop offers a natural ablation: removing Steps A and B reduces it to a pure window-refine loop and isolates the contribution of graph exploration. *Third*,  $L_r$  at any round is a valid prediction, so the round structure exposes a per-round potential that we develop into a training target in the next section.

#### 3.3 A Dense, Recall-Aligned Per-Round Target

#### 3.4 Supervised Training Objective

### 4 Experiments

#### 4.1 Setup

#### 4.2 Main Results

#### 4.3 Does the Coverage Term Help?

#### 4.4 Efficiency and Scaling

### 5 Conclusion

### Limitations

### References

- Toufique Ahmed, Premkumar Devanbu, Christoph Treude, and Michael Pradel. 2024. Can llms replace manual annotation of software engineering artifacts? *arXiv preprint arXiv:2408.05534*.
- Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated unit test improvement using large language models at meta. In *Companion Proceedings of the 32nd ACM International Conference on*

- the Foundations of Software Engineering, pages 185–196. 326
- Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, Sujan Kumar Gonugondla, Hantian Ding, Varun Kumar, Nathan Fulton, Arash Farahani, Siddhartha Jain, Robert Giaquinto, Haifeng Qian, Murali Krishna Ramanathan, and 6 others. 2023. [Multi-lingual evaluation of code generation models](#). In *The Eleventh International Conference on Learning Representations*. 327
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*. 328
- Loubna Ben Allal, Niklas Muennighoff, Lo-gesh Kumar Umapathi, Ben Lipkin, and Leandro von Werra. 2022. A framework for the evaluation of code generation models. <https://github.com/bigcode-project/bigcode-evaluation-harness>. 329
- Shravan Chaudhari, Rahul Thomas Jacob, Mononito Goswami, Jiajun Cao, Shihab Rashid, and Christian Bock. 2025. SpIDER: Spatially informed dense embedding retrieval for software issue localization. *arXiv preprint arXiv:2512.16956*. 330
- Jiayi Chen and 1 others. 2025a. Training software engineering agents and verifiers with SWE-gym. *arXiv preprint arXiv:2412.21139*. 331
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*. 332
- Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025b. [LocAgent: Graph-guided LLM agents for code localization](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8697–8727, Vienna, Austria. Association for Computational Linguistics. 333
- Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubei, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. 2024a. [Introducing SWE-bench Verified](#). Accessed on March 2, 2025. 334
- Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubei, Mia Glaese, Carlos E. Jimenez, John Yang, Kevin Liu, and Aleksander Madry. 2024b. [Introducing SWE-bench Verified](#). OpenAI Blog. 335
- Colin Clement, Dawn Drain, Jonathan Timcheck, Alexey Svyatkovskiy, and Neel Sundaresan. 2020. [PyMT5: multi-mode translation of natural language and python code with transformers](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9052–9065, Online. Association for Computational Linguistics. 336
- Jianbo Dai, Jianqiao Lu, Yunlong Feng, Rongju Ruan, Ming Cheng, Haochen Tan, and Zhijiang Guo. 2024. Mhpp: Exploring the capabilities and limitations of language models beyond basic code generation. *arXiv preprint arXiv:2405.11430*. 337
- Chunyu Deng, Yilun Zhao, Xiangru Tang, Mark Gestein, and Arman Cohan. 2023. Benchmark probing: Investigating data leakage in large language models. In *NeurIPS 2023 Workshop on Backdoors in Deep Learning-The Good, the Bad, and the Ugly*. 338
- Yangruibo Ding, Zijian Wang, Wasi Ahmad, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2024. [CoCoMIC: Code completion by jointly modeling in-file and cross-file context](#). In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 3433–3445, Torino, Italia. ELRA and ICCL. 339
- Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2023. [Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion](#). In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. 340
- Fabio James Fehr, Prabhu Teja S, Luca Franceschi, and Giovanni Zappella. 2025. [CoRet: Improved retriever for code editing](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 775–789, Vienna, Austria. Association for Computational Linguistics. 341
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. [CodeBERT: A pre-trained model for programming and natural languages](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, Online. Association for Computational Linguistics. 342
- Paul Gauthier. 2024. Aider is ai pair programming in your terminal. <https://github.com/paul-gauthier/aider>. 343
- Charles Goodhart. 1976. *Monetary relationships: a view from Threadneedle Street*. University of Warwick. 344

- Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida Wang. 2024. Cruxeval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065*.
- Sumit Gulwani, Oleksandr Polozov, Rishabh Singh, and 1 others. 2017. Program synthesis. *Foundations and Trends® in Programming Languages*, 4(1-2):1–119.
- Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. Graph-codebert: Pre-training code representations with data flow. *Preprint*, arXiv:2009.08366.
- Daya Guo, Canwen Xu, Nan Duan, Jian Yin, and Julian McAuley. 2023. Longcoder: A long-range pre-trained language model for code completion. In *International Conference on Machine Learning*, pages 12098–12107. PMLR.
- Jiafeng Guo, Yixing Fan, Qingyao Ai, and W. Bruce Croft. 2016. A deep relevance matching model for ad-hoc retrieval. CIKM '16, page 55–64, New York, NY, USA. Association for Computing Machinery.
- Jiafeng Guo, Yixing Fan, Liang Pang, Liu Yang, Qingyao Ai, Hamed Zamani, Chen Wu, W. Bruce Croft, and Xueqi Cheng. 2020. A deep look into neural ranking models for information retrieval. *Information Processing & Management*, 57(6):102067.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. 2024. Mlagentbench: evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org.
- Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Live-codebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*.
- Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. 2022. Jigsaw: large language models meet program synthesis. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, page 1219–1231, New York, NY, USA. Association for Computing Machinery.
- Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*.
- Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. Cosil: Software issue localization via llm-driven code repository graph searching. *Preprint*, arXiv:2503.22424.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024a. Can language models resolve real-world github issues? <https://www.swebench.com/>.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024b. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.
- Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. 2024. Dsbench: How far are data science agents to becoming data science experts? *Preprint*, arXiv:2409.07703.
- James A Jones, Mary Jean Harrold, and John Stasko. 2002. Visualization of test information to assist fault localization. In *International Conference on Software Engineering*, pages 467–477.
- Sungmin Kang, Gabin An, and Shin Yoo. 2024. A quantitative and qualitative evaluation of llm-based explainable fault localization. *Proc. ACM Softw. Eng.*, 1(FSE).
- Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-Tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A natural and reliable benchmark for data science code generation. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 18319–18345. PMLR.
- Jia Li, Ge Li, Xuanming Zhang, Yihong Dong, and Zhi Jin. 2024a. Evocodebench: An evolving code generation benchmark aligned with real-world code repositories. *arXiv preprint arXiv:2404.00599*.
- Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Huanyu Liu, Hao Zhu, Lecheng Wang, Kaibo Liu, Zheng Fang, Lanshen Wang, Jiazheng Ding, Xuanming Zhang, Yuqi Zhu, Yihong Dong, Zhi Jin, Binhua Li, Fei Huang, Yongbin Li, Bin Gu, and Mengfei Yang.

2024b. [DevEval: A manually-annotated code generation benchmark aligned with real-world code repositories](#). In *Findings of the Association for Computational Linguistics ACL 2024*, pages 3603–3614, Bangkok, Thailand and virtual meeting. Association for Computational Linguistics.

Ryan Li, Yanzhe Zhang, and Diyi Yang. 2024c. [Sketch2code: Evaluating vision-language models for interactive web design prototyping](#). *Preprint*, arXiv:2410.16232.

Yi Li, Shaohua Wang, and Tien N. Nguyen. 2021. Fault localization with code coverage representation learning. In *International Conference on Software Engineering*.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, and 7 others. 2022. [Competition-level code generation with alphacode](#). *Science*, 378(6624):1092–1097.

Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. [Repobench: Benchmarking repository-level code auto-completion systems](#). In *The Twelfth International Conference on Learning Representations*.

LocAgent Team. 2025. LocAgent: Graph-guided LLM agents for code localization. *arXiv preprint arXiv:2503.09089*. TODO: verify author list.

Qinyu Luo, Yining Ye, Shihao Liang, Zhong Zhang, Yujia Qin, Yaxi Lu, Yesai Wu, Xin Cong, Yankai Lin, Yingli Zhang, Xiaoyin Che, Zhiyuan Liu, and Maosong Sun. 2024. [Repoagent: An llm-powered open-source framework for repository-level code documentation generation](#). *Preprint*, arXiv:2402.16667.

Zexiong Ma, Chao Peng, Qunhong Zeng, Pengfei Gao, Yanzhen Zou, and Bing Xie. 2025. Tool-integrated reinforcement learning for repo deep search. *arXiv preprint arXiv:2508.03012*.

Zohar Manna and Richard J. Waldinger. 1971. [Toward automatic program synthesis](#). *Commun. ACM*, 14(3):151–165.

Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. 2025. Swe-lancer: Can frontier llms earn 1 million from real-world freelance software engineering? *arXiv preprint arXiv:2502.12115*.

Andrew Y Ng, Daishi Harada, and Stuart Russell. 1999. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning*, pages 278–287.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. [Codegen: An open large language](#)

[model for code with multi-turn program synthesis](#). In *The Eleventh International Conference on Learning Representations*.

OrcaLoca Team. 2025. OrcaLoca: An LLM agent framework for software issue localization. *arXiv preprint*. TODO: verify authors and arXiv id.

Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2025. [Repograph: Enhancing AI software engineering with repository-level code graph](#). In *The Thirteenth International Conference on Learning Representations*.

Mike Papadakis and Yves Le Traon. 2015. Metallaxis-FL: Mutation-based fault localization. *Software Testing, Verification and Reliability*, 25(5-7):605–628.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*.

Muhammad Shihab Rashid, Christian Bock, Yuan Zhuang, Alexander Buchholz, Tim Esler, Simon Valentin, Luca Franceschi, Martin Wistuba, Prabhu Teja Sivaprasad, Woo Jung Kim, Anoop Deoras, Giovanni Zappella, and Laurent Callot. 2025. [Swe-polybench: A multi-language benchmark for repository level evaluation of coding agents](#). *Preprint*, arXiv:2504.08703.

Revanth Gangi Reddy, Ye Liu, Wenting Zhao, JaeHyeok Doo, Tarun Suresh, Daniel Lee, Caiming Xiong, Yingbo Zhou, Semih Yavuz, and Shafiq Joty. 2025a. SweRank+: Multilingual, multi-turn code ranking for software issue localization. *arXiv preprint arXiv:2512.20482*.

Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. 2025b. [Swerank: Software issue localization with code ranking](#). *Preprint*, arXiv:2505.07849.

Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. 1994. [Okapi at trec-3](#). In *Text Retrieval Conference*.

Moshe Shaked and J. George Shanthikumar. 2007. *Stochastic Orders*. Springer.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Haiyang Shen, Yue Li, Desong Meng, Dongqi Cai, Sheng Qi, Li Zhang, Mengwei Xu, and Yun Ma. 2025. [Shortcutsbench: A large-scale real-world benchmark for api-based agents](#). *Preprint*, arXiv:2407.00132.

- Atefeh Sohrabizadeh, Jialin Song, Mingjie Li, Rajarshi Roy, Chankyu Lee, Jonathan Raiman, and Bryan Catanzaro. 2025. *Nemotron-CORTEXA: Enhancing LLM agents for software engineering tasks via improved localization and solution diversity*. In *Forty-second International Conference on Machine Learning*.
- Stack Overflow. 2023. Stack overflow developer survey 2023. <https://survey.stackoverflow.co/2023/#most-popular-technologies-language-prof>.
- Tarun Suresh, Revanth Gangi Reddy, Yifei Xu, Zach Nussbaum, Andriy Mulyar, Brandon Duderstadt, and Heng Ji. 2025. *CoRNStack: High-quality contrastive data for better code retrieval and reranking*. In *The Thirteenth International Conference on Learning Representations*.
- SweRank Team. 2025. SweRank: Software issue localization with code ranking. *arXiv preprint arXiv:2505.07849*. TODO: verify author list and title.
- Marc Szafraniec, Baptiste Roziere, Hugh James Leather, Patrick Labatut, Francois Charton, and Gabriel Synnaeve. 2023. *Code translation with compiler representations*. In *The Eleventh International Conference on Learning Representations*.
- Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Hui Haotian, Liu Weichuan, Zhiyuan Liu, and Maosong Sun. 2024. *DebugBench: Evaluating debugging capability of large language models*. In *Findings of the Association for Computational Linguistics ACL 2024*, pages 4173–4198, Bangkok, Thailand and virtual meeting. Association for Computational Linguistics.
- Mark Vero, Niels Mündler, Victor Chibotaru, Veselin Raychev, Maximilian Baader, Nikola Jovanović, Jingxuan He, and Martin Vechev. 2025. *Baxbench: Can llms generate correct and secure backends?*
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2025. *Openhands: An open platform for AI software developers as generalist agents*. In *The Thirteenth International Conference on Learning Representations*.
- Xingyao Wang and 1 others. 2024. *OpenHands: An open platform for AI software developers as generalist agents*. *arXiv preprint arXiv:2407.16741*.
- Zhiruo Wang, Grace Cuenca, Shuyan Zhou, Frank F Xu, and Graham Neubig. 2022. *Mconala: a benchmark for code generation from multiple natural languages*. *arXiv preprint arXiv:2203.08388*.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. 2025. *SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution*. *arXiv preprint arXiv:2502.18449*.
- Yonghao Wu, Zheng Li, Jie M. Zhang, Mike Papadakis, Mark Harman, and Yong Liu. 2023. *Large language models in fault localisation*. *Preprint*, arXiv:2308.15276.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024a. *Agentless: Demystifying LLM-based software engineering agents*. *arXiv preprint arXiv:2407.01489*.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024b. *Agentless: Demystifying llm-based software engineering agents*. *Preprint*, arXiv:2407.01489.
- Yiqing Xie, Alex Xie, Divyanshu Sheth, Pengfei Liu, Daniel Fried, and Carolyn Rose. 2024. *Codebenchgen: Creating scalable execution-based code generation benchmarks*. *arXiv preprint arXiv:2404.00566*.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024a. *SWE-agent: Agent-computer interfaces enable automated software engineering*. *arXiv preprint arXiv:2405.15793*.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024b. *Swe-agent: Agent-computer interfaces enable automated software engineering*. *arXiv preprint arXiv:2405.15793*.
- John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida I. Wang, and Ofir Press. 2024c. *Swe-bench multimodal: Do ai systems generalize to visual software domains?* *Preprint*, arXiv:2410.03859.
- Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. *Codereval: A benchmark of pragmatic code generation with generative pre-trained models*. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA. Association for Computing Machinery.
- Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. 2022. *Cert: continual pre-training on sketches for library-oriented code generation*. *arXiv preprint arXiv:2206.06888*.
- Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. 2025. *Multi-swe-bench: A multilingual benchmark for issue resolving*. *Preprint*, arXiv:2504.02605.

Daoguang Zan, Zhirong Huang, Ailun Yu, Shaolin Lin, Yifan Shi, Wei Liu, Dong Chen, Zongshuai Qi, Hao Yu, Lei Yu, Dezhi Ran, Muhan Zeng, Bo Shen, Pan Bian, Guangtai Liang, Bei Guan, Pengjie Huang, Tao Xie, Yongji Wang, and Qianxiang Wang. 2024. *Swe-bench-java: A github issue resolving benchmark for java*. Preprint, arXiv:2408.14354.

Dejiao Zhang, Wasi Uddin Ahmad, Ming Tan, Hantian Ding, Ramesh Nallapati, Dan Roth, Xiaofei Ma, and Bing Xiang. 2024a. *CODE REPRESENTATION LEARNING AT SCALE*. In *The Twelfth International Conference on Learning Representations*.

Dejiao Zhang, Wasi Uddin Ahmad, Ming Tan, Hantian Ding, Ramesh Nallapati, Dan Roth, Xiaofei Ma, and Bing Xiang. 2024b. *CODE REPRESENTATION LEARNING AT SCALE*. In *The Twelfth International Conference on Learning Representations*.

Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024c. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. *arXiv preprint arXiv:2401.07339*.

Yuge Zhang, Qiyang Jiang, Xingyu Han, Xingyu Han, Nan Chen, Yuqing Yang, and Kan Ren. 2024d. *Benchmarking data science agents*. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5677–5700, Bangkok, Thailand. Association for Computational Linguistics.

Zhaoxi Zhang, Yitong Duan, Yanzhi Zhang, Yiming Xu, Zhixiang Wang, Kun Liang, Yang Li, Jiahui Liang, Deguo Xia, Jizhou Huang, Jiyan He, and Yunfang Wu. 2025. One tool is enough: Reinforcement learning for repository-level LLM agents. *arXiv preprint arXiv:2512.20957*.

Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, and 1 others. 2024. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, and 1 others. 2024. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*.

Albert Örwall. 2024. *Moatless tools*.

## A Proof of the Telescoping Identity

## B Code Graph Construction

The scaffolding’s neighborhood expansion (Section 3.2, Step B) walks a per-repository code graph  $\mathcal{G}=(V, E)$ . We follow the four-edge schema of

Chen et al. (2025b) as implemented in Chaudhari et al. (2025), summarised here for completeness.

For each repository the graph is built from the source files written in the repository’s primary language; files in secondary languages are ignored. Nodes represent code entities at four granularities (directory, file, class, and function), and edges encode four structural relations: a *contains* edge linking each container to the code units it declares, an *invokes* edge between a function and each function it calls, an *imports* edge between a file and each module it imports, and an *inherits* edge between a class and each parent class it extends. Every non-directory node stores its source text alongside its identifier, so the model can see code when a node is shown.

We extract Python source files with the standard `ast` module and Java, JavaScript, and TypeScript files with `Tree-sitter`<sup>1</sup>. The traversal of the repository’s directory tree adds directory nodes for navigation, skips files that contain no class or function definition, and emits one node per declared entity together with its containment, call, import, and inheritance edges. Multi-file class definitions and the more flexible declaration patterns of the non-Python languages are handled with language-specific `Tree-sitter` queries; we refer to Chaudhari et al. (2025) for the full list of cases. Graphs are constructed on demand at the start of a localization run and reused across the rounds.

For the experiments in Section 4, we use the graphs released with Chaudhari et al. (2025) for SWE-PolyBench and SWE-Bench Verified, and construct graphs from scratch for the SWE-Bench Lite dev split with the same pipeline.

## C Scaffolding and Training Details

## D Additional Results

## E AI Disclosure

<sup>1</sup><https://github.com/tree-sitter>